When More Cores Hurts: The Vector Database Scaling Paradox in HPC

Seth Ockerman[1], Song Young Oh[2], Amal Gueroudji[3], Rochana Chaturvedi[3], Philip Carns[3], Nicholas Chia[3], Matthieu Dorier[3], Robert Latham[3] Tanwi Mallick[3], Swan Perarnau[3], Robert Underwood[3], Kyle Chard[2,3], Ian Foster[3,2], Robert Ross[3], Shivaram Venkataraman[1]

[1]University of Wisconsin–Madison [2]University of Chicago [3]Argonne National Laboratory

***Abstract*—Vector databases have been designed and optimized for cloud environments; however, emerging scientific AI workloads (e.g., molecular search, meteorological trajectory detection, and literature-driven hypothesis generation) demand efficient, scalable execution on HPC systems. We present a large-scale evaluation of three state-of-the-art vector databases—Qdrant, Milvus, and Weaviate—on two production supercomputers, scaling to 256 distributed workers across 64 compute nodes. We evaluate representative workload patterns—mixed read/write and write-then-read—using popular benchmarks, multimodal embeddings, and a novel real-world scientific dataset. Our results reveal that workload characteristics can limit latency reduction, additional cores can reduce query throughput by up to 30.67%, and scaling from 16 to 256 workers (16×) only yields a 5.46× improvement. This scaling paradox exposes the fundamental mismatch between cloud-oriented designs and HPC systems, highlighting the need for new, HPC-aware vector database designs.**

***Index Terms*—Vector Databases, High-Performance Computing, Performance Evaluation, Distributed Systems**

I. Introduction

Vector databases (VDBs) have emerged as a crucial tool for managing and querying unstructured data. VDBs store compact vector representations of data known as embeddings [1], [2] and facilitate efficient semantic search, powering applications such as recommendation systems [3], [4], retrieval-augmented generation [5]–[7], and long-term agentic memory [8]–[11]. These traditional AI workloads, as well as emerging scientific use cases such as cosmology [12], earth science [13], and medicine/biology [14]–[17], require scalable VDB performance on high-performance-computing (HPC) systems. Scientific datasets are increasingly large, already reaching petabyte scale [18]–[20]; and, concurrently, researchers are investigating representing these datasets as embeddings for semantic discovery and AI use [21]–[23]. These developments underscore the need for the HPC community to understand what is required to operate VDBs at HPC scale.

VDB development has been driven primarily by industry [24]–[29], with a focus on cloud-based deployments [30]. As a result, to the best of our knowledge, little prior work has investigated, or optimized, performance of VDBs on HPC systems. HPC systems differ significantly from typical cloud platforms in terms of performance [31]–[33] and architecture [34]–[36], featuring many-core CPUs, multiple high-memory GPUs per node, a tightly coupled architecture connected by a high-bandwidth network, and a deep storage hierarchy [37]. This gap motivates our study, which evaluates the behavior of state-of-the-art VDBs on HPC systems and examines whether their designs align with the requirements of real-world scientific workloads.

We evaluate three popular VDBs—Weaviate [28], Milvus [24], and Qdrant [26]—using four embedding/query datasets (Pes2o-VE, Yandex-text-to-image [38], GIST [39], and dbpedia-openai-1M [40]) on two production supercomputers. To support our evaluation and promote reproducibility, we release `VECHINI`,[1] a configurable framework for benchmarking VDBs on HPC platforms. Additionally, we construct and release Pes2o-VE,[2] a real-world embedding dataset inspired by a biological application, which, to the best of our knowledge, is among the largest publicly available embedding datasets (≈88 million embeddings or 843.56 GB). Using VECHINI, we study two common VDB workload patterns: (i) insert-then-query, representative of scientific and knowledge-driven workflows such as hypothesis generation and literature retrieval, and (ii) mixed insert-query, characteristic of emerging AI and agentic applications [41], [42]. Our study provides the following valuable insights into the performance of current VDBs on HPC systems:

Cloud vs HPC: We demonstrate that, compared with the cloud, query throughput and index time improve the most on HPC systems, while already low-latency queries see minimal improvement.

Impact of Embedding geometry: Complex embedding spaces that require deeper search to meet recall targets [43]–[45] benefit more from HPC architectures, achieving greater reductions in latency and improvements in throughput.

Limits of the Streaming-based Insertion Model: VDB architectures fail to exploit the hierarchical storage landscape of HPC (e.g., local SSD, DAOS, Lustre) and enforce fine-grained consistency even during bulk ingestion, leaving substantial resources unused.

Hidden Costs at Scale: VDB systems incur write-amplification, leading to significant storage overhead (2.8× with Milvus) that may hinder scalability for large datasets.

Impact of Segmentation: Traditionally, recall–cost trade-offs are tuned through index parameters; in modern VDBs,

[1]https://github.com/OckermanSethGVSU/VECHINI

[2]https://www.materialsdatafacility.org/detail/e31e3225-7f75-4313-9983-f8b75811405f-1.0

however, the system's data partitioning strategy also acts as an implicit tuning axis, jointly shaping recall, throughput, and latency (e.g., 1 vs. 8 partitions improved recall from 0.962 to 0.985 but increased search time from 4.19s to 22.65s.)

Limited or Inverse Scaling: Across both single-node and distributed settings, adding cores or nodes yields diminishing returns in query throughput and, in some cases, even degrades query performance by up to 30.67%.

In summary, we study multiple state-of-the-art VDBs with up to 256 distributed VDB workers across 64 compute nodes, exploring both intra- and internode parallelism. We examine how VDBs interact with key HPC architectural features, including many-core CPUs, GPU acceleration, and DAOS/Lustre storage [34], [46]. Using our experience, we provide practical insights that aid scientists in using existing VDBs on HPC systems and highlight key areas for new research.

II. Background

VDBs are specialized databases designed for efficient search over vectorized data representations known as embeddings, where embeddings encode semantic features about the data [47], [48]. To perform a search, VDBs compute distances between a query vector and stored embeddings to identify the top-k nearest candidates, a process known as nearest-neighbor search [49]–[51]. As the number of embeddings grows, exhaustive comparison becomes infeasible; instead, VDBs use approximate-nearest-neighbor (ANN) search [52] with index structures [53]–[57] that provide controllable accuracy–latency trade-offs. Accuracy is measured primarily by `recall@k` (see Equation (1)), defined as the fraction of the k true nearest neighbors returned by a top-k ANN query.

A. Graph-Based Indices

Graph-based indices [58]–[60], especially hierarchical navigable small world (HNSW) graphs [61], are the most popular choices for VDB indexing. HNSW graphs provide strong performance and the ability to preserve neighborhood structure in high-dimensional spaces. To control the trade-off between latency and accuracy, HNSW uses three key parameters: M, $ef_{\text{construction}}$ (EF-C), and ef_{Search} (EF-S). M controls the maximum node connectivity, while EF-C and EF-S refer to the size of the candidate queue used during index construction and search, respectively. The latter two parameters act as a proxy for index/search effort, with higher values indicating a more exhaustive process.

$$\text{Recall@}k = \frac{|Results_k \cap GroundTruth_k|}{k} \tag{1}$$

B. Distributed Vector Databases

While HNSW and more recent GPU-based graph indices [60], [62] provide strong performance on a single node, their effectiveness is ultimately bounded by node-level resource constraints. HPC workloads, by contrast, operate at scales that exceed the capabilities of any single node. To reflect the requirements of HPC environments, we restrict our evaluation to VDBs that support distributed computation.

While Milvus, Weaviate, and Qdrant differ architecturally, they share several core design patterns: a broadcast–gather query model, consistent hashing for write routing [63], and segmented data layouts. In order to support queries over distributed indices, queries are broadcast to all relevant workers, which execute ANN search locally The local results are then aggregated into a global result. For new data, consistent hashing routes inserts to shards and appends the data to in-memory open segments. Once a segment reaches a predefined size threshold, it is marked as eligible for indexing. At this point, Milvus flushes the segment to storage for later indexing, whereas Qdrant and Weaviate build the segment index directly in memory. This distinction reflects a broader architectural divide. Qdrant and Weaviate use a worker-centric design in which each node manages the full life cycle of its data (insertion, indexing, query). In contrast, Milvus decomposes responsibilities across microservices and explicitly separates compute/storage. The front-ends of Milvus are proxies, which route data to the correct component and perform the final level of query aggregation. To process incoming data, Milvus uses streaming nodes, which maintain open segments in memory, perform exhaustive search over unindexed data as needed, and flush segments to durable storage (typically MiniIO or remote S3) when they reach a size threshold. For indexing and querying, Milvus utilizes data and query nodes, respectively, with both loading the relevant segments from durable storage into their local memory.

C. Related Work

Vector databases are an active area of research, with multiple survey and evaluation efforts. Several recent works [64]–[67] summarize key challenges, design choices, and opportunities in vector database systems; however, they do not provide empirical evaluations. Most empirical evaluations, across both academia [68]–[72] and industry [73]–[75], focus on centralized, single-node VDB systems with a single index rather than on distributed systems. While a growing body of work is focused on improving distributed VDBs [76]–[79], these studies evaluate query-time performance in isolation, rather than the full workload life cycle—spanning insertion, indexing, and querying—or the properties of the distributed cloud systems on which they are deployed. In scientific workflows such as large-scale simulations [80], [81], data is continually generated and can reach petabyte scale. In this setting, insertion and indexing performance are critical to determine whether the system can keep pace with data generation, highlighting the need to move beyond query-time performance alone.

Recent work [30] examines the performance characteristics of two index types in a cloud environment. The results highlight the impact of cloud storage and limited network throughput on search performance and index design. This work aims to uncover similar insights; however, we focus on HPC environments and evaluate three distributed vector database systems end-to-end, rather than comparing individual index structures. Recent work [15] evaluates Qdrant on an HPC system. However, it considers only a single system and VDB,

TABLE I: Datasets used in this study listed from smallest to largest. All vectors are stored using `float32` representations.

Dataset	# Embeddings	Dim	Total Size (GB)	Use Case	Model(s)	Modality
GIST	1,000,000	960	3.58 GB	Image Search	N/A	Image
dbpedia-openai-1M	1,000,000	1536	5.72 GB	Information Retrieval	Text-embedding-ada-002	Text
Yandex-text-to-image	1,000,000,000	200	745.05 GB	Multimodal Information Retrieval	Se-ResNext-101, DSSM	Image, Text
Pes2o-VE	88,453,763	2560	843.56 GB	Academic Corpus Review	Qwen3-Embedding-4B	Text

TABLE II: Overview of Polaris & Aurora compute nodes.

Component	Polaris	Aurora
CPU	32 cores	104 cores
Memory	512 GB DDR4	1 TB DDR5
GPUs	4×40 GB NVIDIA A100	6×128 GB Intel Max
Node-local storage	512 GB SSD	N/A
Network	2×25 GB/s NIC	8×25 GB/s NIC
Networked Storage	Lustre PFS	Lustre PFS & DAOS

does not exercise advanced features such as GPU acceleration, and does not explore HPC-specific architectural components (e.g., DAOS or Lustre). This work provides a more systematic evaluation across multiple HPC systems, distributed VDB implementations, and datasets, including Pes2o-VE, which is significantly larger than datasets used in prior VDB workload studies [15], [30], [72].

III. Methodology

We conduct our evaluation on two HPC systems: Polaris,[3] which is equipped with NVIDIA A100 GPUs, and Aurora,[4] which is equipped with Intel PVC GPUs. For scaling experiments, we default to Aurora, which offers a newer system architecture and greater overall computational capacity. GPU-based evaluations are performed exclusively on Polaris because of limited support for Intel GPUs in the evaluated VDBs. An architectural overview of both HPC systems is provided in Table II. Cloud deployments of VDBs typically rely on Docker and Kubernetes, which are not supported on most HPC systems, including Polaris and Aurora. Instead, we use Apptainer,[5] which converts Docker images into an HPC-compatible format, with Qdrant 1.16.2, Milvus 2.6.6, and Weaviate 1.36.0.

Milvus is designed as a cloud-native distributed VDB that assumes shared persistent state via object storage (e.g., S3/MinIO) or local disk in single-node deployments. However, based on recommendations from Milvus developers,[6] we instead adopt an HPC-oriented deployment that replaces MinIO with Lustre. In this configuration, all components directly access Lustre as a shared filesystem. To our knowledge, this setup is not publicly documented and represents a novel deployment approach for HPC environments.

We test each client implementation (e.g., Python, Go, Rust) and report only the results with the optimal client for brevity (Go for Weaviate and Milvus, and Rust for Qdrant). In this context we use the term "worker" to refer to an independent VDB node. However, Milvus relies on a set of microservices; for single-worker experiments, we use the closest equivalent: Milvus Standalone, a single-process deployment of its architecture. During distributed testing, we place up to four workers per compute node to balance parallelism with node-level resource limits. Additionally, note that in all cases, clients are placed on a separate compute node from the VDB instance(s) to measure network performance and prevent resource contention.

A. VECHINI

To support reproducible VDB evaluation in HPC environments, we design `VECHINI`: **V**ector Database **E**valuation and **C**haracterization for **H**PC **I**nsertion, Indexing, and **N**earest-neighbor Search. Existing VDB benchmarking frameworks [70], [73]–[75], [82] assume Docker- or Kubernetes-based orchestration, which is unavailable on many HPC systems. VECHINI instead uses HPC-native tools, including Apptainer and MPI, enabling practitioners to deploy VDBs within the security constraints of their HPC platforms. Using a schema-driven configuration layer, VECHINI generates experiments for common VDB tasks, including insertion, indexing, and querying. Experiments are defined using configuration files that the benchmark expands into experiment directories for submission to the HPC job scheduler. By modifying the configuration file, users can control key HPC features such as cores-per-worker, number of compute nodes, clients/workers-per-compute-node, and storage backend without managing system-specific implementation details. VECHINI currently supports Qdrant, Milvus, and Weaviate on the Polaris and Aurora systems; however, the framework is designed to be extensible, allowing users to add new VDBs and HPC platforms while reusing the same schema-driven configuration layer.

B. Embedding Datasets

Thoroughly evaluating VDBs requires accounting for varied embedding characteristics and dataset scale, as these jointly determine system behavior and performance [83], [84]. Accordingly, we select two distinct small-scale embedding datasets, GIST [39] and dbpedia-openai-1M [40], and two large-scale embedding datasets, Yandex-text-to-image (Yandex-T2I) and Pes2o-VE. GIST consists of 960-dimensional embeddings derived from multiple image collections, while dbpedia-openai-1M [40] is composed of 1560-dimensional embeddings generated from text excerpts by the text-embedding-ada-0002 [85] model.

To expand our evaluation to larger datasets, we include Yandex-T2I [38], a 745 GB multimodal dataset for cross-modal retrieval. Additionally, we introduce

[3]https://www.alcf.anl.gov/polaris

[4]https://www.alcf.anl.gov/aurora

[5]https://apptainer.org/

[6]https://github.com/milvus-io/milvus/discussions/48684#discussioncomment-16427291

TABLE III: Cloud vs HPC vector database performance with a recall minimum of 0.95. All reported cloud results were obtained by a prior publicly available benchmarking effort by Qdrant [74] (denoted using "*cloud*" in the platform column). HPC results are the median value obtained from 3 runs. Note that Weaviate did not use deferred indexing in the original experiments and instead built the index during data ingestion, resulting in an index time of 0.

VDB	Platform	Dataset	M	EF-C	EF-S	Recall	Upload Time	Index Time	QPS	Latency
Milvus	*Cloud*	GIST	32	128	128	0.964	84.968 (s)	487.061 (s)	281.53	322.63 (ms)
	Aurora	GIST	32	128	128	0.961	197.29 (s)	84.50 (s)	1470.29	23.85 (ms)
	Cloud	dbpedia-openai	32	128	128	0.998	176.66 (s)	886.152 (s)	154.06	582.51 (ms)
	Aurora	dbpedia-openai	32	128	128	0.996	129.03 (s)	205.41 (s)	799.50	46.87 (ms)
Qdrant	*Cloud*	GIST	32	512	256	0.960	146.29 (s)	2014.81 (s)	433.95	207.28 (ms)
	Aurora	GIST	32	512	256	0.980	45.16 (s)	215.22 (s)	1373.26	58.31 (ms)
	Cloud	dbpedia-openai	16	128	64	0.967	238.42 (s)	671.83 (s)	1260.53	3.26 (ms)
	Aurora	dbpedia-openai	16	128	64	0.979	62.41 (s)	85.12 (s)	1738.55	3.24 (ms)
Weaviate	*Cloud*	GIST	64	512	512	0.970	836.96 (s)	N/A	496.17	184.37 (ms)
	Aurora	GIST	64	512	512	0.988	281.84 (s)	N/A	2481.86	30.72 (ms)
	Cloud	dbpedia-openai	64	512	64	0.975	836.96 (s)	N/A	1142.13	4.99 (ms)
	Aurora	dbpedia-openai	64	512	64	0.986	395.53 (s)	N/A	1518.28	4.87 (ms)

`Pes2o-vector-embeddings` (Pes2o-VE). Pes2o-VE is motivated by a biology application that augments BV-BRC [86], a comprehensive bioinformatics resource, with context retrieved from an academic corpus. Using 22,723 genome-related terms, the VDB is queried for relevant passages, which—along with structured records—are provided to a scientific reasoning model to generate structured "bio-narratives" for a downstream RAG pipeline. Pes2o-VE is generated from the Pes2o text corpus [87] (8M+ academic papers) using a combination of recursive splitting and semantic chunking with the Qwen3-Embedding-4B model [47]. The resulting dataset contains 88M embeddings (843.56 GB), comparable in scale to many of the largest available embedding benchmarks (e.g., Yandex-T2I, LAION-400M [88]).

C. Selecting Representative VDB Workload Patterns

We consider two common patterns observed in VDB workloads: insert-then-query (a bulk write phase followed by a sustained read phase) and mixed insert-query (interleaved read and write operations). The former pattern is common to knowledge retrieval tasks such as documentation search [89], case-law review [90], [91], and traditional RAG pipelines [12], [17], [92]–[94], while the latter is characteristic of emerging agentic workloads [95], [96] and online-content recommendation systems [97]. Notably, in practice, even mixed insert-query workloads typically begin with an initial phase that inserts background knowledge (e.g., relevant scientific literature for an agent's task), highlighting the importance of the initial ingestion step.

D. Experimental Metrics

Throughout our experiments, we use the following terminology to describe key aspects of VDB workloads:

- **Upload (insert) time**: Time to transfer data from client(s) to the VDB, measured as time to "searchability," namely, when newly inserted data becomes visible to query.
- **Index time**: Time to construct the index over all data and reach a steady state with no further indexing.
- **Queries per second (QPS)**: Number of queries completed per second.
- **Query latency**: End-to-end time to complete a query or query batch.
- **P95/99 latency**: Latency below which a given threshold (95/99%) of query or query batches complete, capturing tail behavior.
- **Recall**: The fraction of true nearest neighbors retrieved relative to exhaustive search (see Equation (1)).

IV. From Cloud to HPC: Performance Impact

To study key VDB performance differences between cloud and HPC settings, we execute an open-source cloud benchmark suite [74] on Aurora. We employ identical parameters and versions as stated in the published benchmark results. [7] The original experiments were conducted using two nodes: a client node with 8 vCPUs, 16 GiB of memory, and 64 GiB of storage running 100 Python clients; and a server node for the VDB with 8 vCPUs, 32 GiB of memory, and 64 GiB of storage. During execution, each Python client sends sequential non-batched queries, operating in parallel. We mimic this deployment model, replacing the cloud instances with Aurora compute nodes described in Table II. To select configurations for testing, we filter by a minimum recall threshold, using a cutoff of 0.95 recall to reflect the original constraints of the benchmark. For settings that meet the minimum recall requirement, we sort configurations by QPS and select the best-performing configuration.

A. Imbalanced Impact of HPC Resources

As shown in Table III, the benchmarks exhibit markedly different performance characteristics in cloud environments than in HPC, with all VDBs exhibiting substantial increases in QPS (1.33×–5.22×) and corresponding reductions in latency (1.01×–13.53×). Interestingly, we observe diminishing returns for latency, where beyond a certain point, additional computational resources no longer meaningfully reduce response

[7] https://github.com/qdrant/vector-db-benchmark/tree/c5b4d45659feafaaa968b6f07fdc12a7eb20e171

time. The dbpedia-openai results for Qdrant and Weaviate illustrate this effect clearly: although QPS increases on HPC hardware, individual query latency remains largely unchanged. Additionally, the magnitude of improvement varies across VDB systems. Milvus, in particular, demonstrates the largest relative gains on HPC hardware across both datasets, reducing latency by an average of 12.98× and increasing throughput by an average of 5.21×.

Beyond query serving, HPC deployments also accelerate data ingestion and index construction. On average, upload time decreases by 2.32×, while index construction time decreases by an average of 6.83×. Notably, index build time exhibits the largest absolute reduction in execution time. This aligns with expectations as index construction is an inherently compute- and memory-intensive process, during which we typically observe sustained CPU utilization of 90%+. Consequently, HPC systems, with their higher core counts and increased memory bandwidth, are particularly well suited to accelerating index construction. In the majority of cases, upload time also decreases on HPC architectures; however, the gains are more modest (see Section V for a root cause analysis). One notable outlier is the increase in upload time for GIST when using Milvus; additional HPC runs revealed variability in Milvus's GIST upload time (75–289 seconds).

B. Embedding Geometry and HPC Performance

The benefit of HPC resources differs between the two datasets, with larger latency and QPS improvements observed for GIST. Prior work [43]–[45] has shown that a metric known as intrinsic dimensionality (ID) [98] can serve as a proxy for search difficulty. ID measures the underlying complexity of the embedding space by estimating the minimum number of dimensions needed to describe its structure, where a higher ID indicates a more challenging search space. Analysis reveals that GIST has a higher ID (46.85) compared with dbpedia-openai (31.32). As a result, GIST requires a more intensive search to achieve 0.95 recall with Weaviate and Qdrant, thus benefiting more from the increased resources present in HPC.

Our results reveal that VDB performance differs substantially between cloud and HPC systems. We observe uneven scaling across workflow stages and an interplay between acceleration and embedding geometry. These results expose fundamentally different performance properties, demonstrating that prior cloud-based evaluations do not directly generalize to HPC.

Lesson 1: Indexing time and QPS significantly improve on HPC compared with the cloud, while already low-latency values remain unchanged.

Lesson 2: Complex embedding spaces require greater search effort, increasing sensitivity to available compute and amplifying the impact of HPC resources.

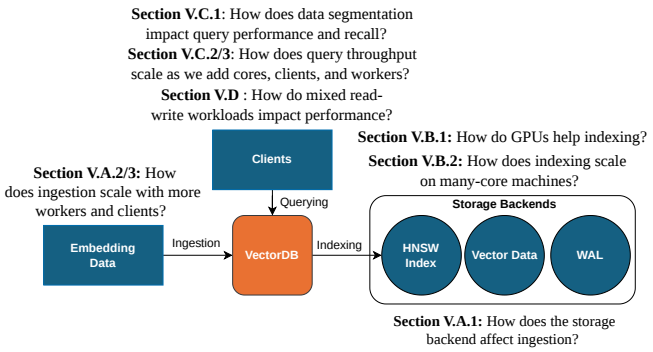

Fig. 1: Life cycle of a vector database.

V. Evaluating the VDB Life Cycle on HPC Systems

In the absence of VDB-focused studies on HPC systems, the community lacks a baseline for expected state-of-the-art performance and an understanding of how existing designs' strengths and weaknesses manifest. To address this gap, we evaluate two representative workload patterns that span the spectrum of VDB usage in both industry and science: *insert-then-query* [12], [17], [89] and *mixed insert-query* [42], [95], [96]. Figure 1 shows the typical life cycle of a VDB, which consists of data ingestion (upload), indexing, and querying, and key questions that we seek to answer in each section. Using the two workload patterns, we characterize how existing VDBs leverage HPC resources—such as multitier storage systems and GPU acceleration—and how they scale in many-core and distributed environments throughout their life cycle.

A. Insertion

In this section we evaluate the first stage of the VDB life cycle: ingestion/upload. Section V-A1 examines how the underlying storage medium impacts performance, testing whether VDBs can effectively leverage HPC's multitier storage hierarchy. Section V-A2 evaluates the insertion limits of a single VDB worker to identify system-level bottlenecks that constrain throughput. Section V-A3 explores distributed insertion scalability to determine whether existing VDBs can meet the demands of large-scale scientific datasets.

In a bulk upload scenario, queries occur after ingestion and indexing. To improve upload performance and align with developer recommendations,[8] we defer HNSW construction until after upload completion. During upload, all systems use a flat index (i.e., exhaustive search if queried). To measure "searchability" (see Section III-D), we utilize system-specific mechanims. For Milvus, we issue a sentinel query that succeeds once all entities are queryable; for Weaviate, we measure when the asynchronous indexing queue becomes empty; and for Qdrant, we use the count API with `exact=true` to confirm that all vectors are visible. For parity, we use a single shard per worker for all VDBs. To focus on large-scale, high-dimensional data characteristics of modern embeddings [99]–[101], we limit upload experiments to Pes2o-VE as a representative HPC embedding workload.

[8] https://qdrant.tech/documentation/tutorials-develop/bulk-upload/

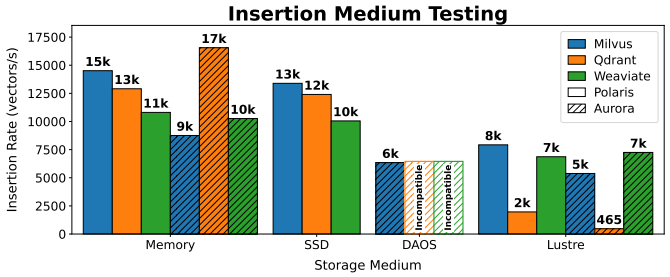

Fig. 2: Pes2o-VE: Testing the impact of different storage mediums on insertion rate on Polaris and Aurora.

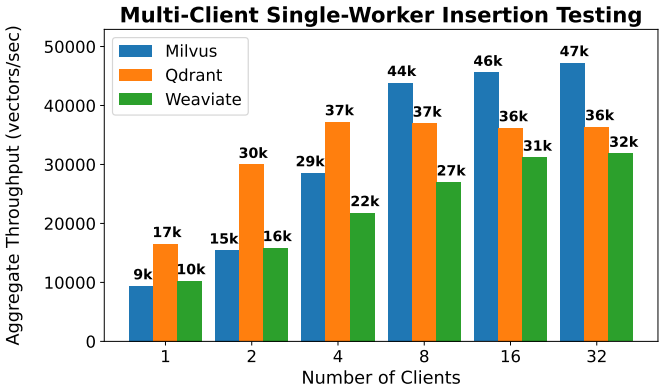

Fig. 3: Pes2o-VE: testing single-worker multiclient insertion on Aurora.

1) Impact of Storage Medium on Performance: To understand the interaction between storage medium and insertion performance, we evaluate a variety of HPC storage backends using a single-client, single-worker setup. We sweep upload batch sizes from 32 to 32,768 vectors, selecting the best-performing option. We test each system's supported storage backends: memory, SSD, and Lustre on Polaris and memory, DAOS, and Lustre on Aurora. To ensure computability with DAOS, we use the DFUSE POSIX interface [34]. We exclude Qdrant and Weaviate from DAOS experiments because several internal components of both VDBs utilize mmap system calls that are not supported by DAOS.

As shown in Figure 2, the underlying storage medium has a substantial impact on achievable insertion throughput. Across all three VDBs, in-memory storage delivers the highest performance, followed by local SSD on Polaris and DAOS on Aurora, with Lustre consistently exhibiting the lowest throughput. Although Lustre is not designed for the small, frequent writes typical of ingestion, it remains necessary in situations that require long-term durability and significant storage capacity. Notably, DAOS achieves insertion rates approaching those of main memory with Milvus (8758 vectors/s vs 6432 vectors/s), highlighting its promise as an HPC-optimized persistence layer.

As shown in Figure 2, Qdrant insertion throughput drastically falls on Lustre compared with the other VDBs. Qdrant's performance is caused by the interaction between storage latency and its update pipeline. While all VDBs write to a write-ahead log (WAL) before asynchronously updating segments, Qdrant requires each insert to reserve a slot in a bounded update queue before completing the WAL write. This queue is drained by a thread applying segment mutations. Initially, reservation latency is negligible (0.000 ms); but because of slow segment mutation (1011 ms), the thread cannot drain the queue quickly enough. This results in significant queuing delay, with reservation latency rising to 1445 ms. In contrast, Lustre's latency impacts only a small portion of the critical write path in Milvus and Weaviate—primarily WAL persistence—resulting in lower performance degradation.

2) Single-Worker Insertion: To assess per-worker ingest scalability, we increase concurrent upload clients until performance saturates on Aurora. As shown in Figure 3, Qdrant achieves the highest per-client throughput (37,218 vectors/s with four clients) but does not scale, with minimal gains beyond four clients due to severe WAL contention (104.034 ms waiting vs. 3.746 ms writing). Weaviate trails just behind Qdrant, peaking at 31,859 vectors/s with 32 clients. Milvus reaches a higher peak throughput (47,236 vectors/s at 32 clients) before declining, as insertion becomes dominated by the WAL append pipeline (67.3 ms of 75.0 ms), which saturates at higher concurrency. To demonstrate the impact of increased WAL parallelism, we modify Qdrant's and Milvus's configurations to increase the number of shards, focusing on Qdrant and Milvus because they achieved the highest insertion throughput. With 32 clients and WALs, Qdrant and Milvus achieve throughputs of 223,843 and 138,860 vectors/second, respectively. These results highlight the importance of WAL parallelism as a mechanism to increase resource utilization on HPC systems.

3) Multiworker Insertion: To support large-scale scientific datasets, VDBs must support efficient parallel ingestion. We evaluate multinode insert scaling by increasing distributed workers from 1 to 32 (powers of two). We use the optimal batch size and clients per worker from Section V-A2, evenly partitioning data across workers to maximize performance. For Milvus, we utilize the HPC configuration described in Section III and scale the components responsible for initial data ingestion: proxies and streaming nodes.

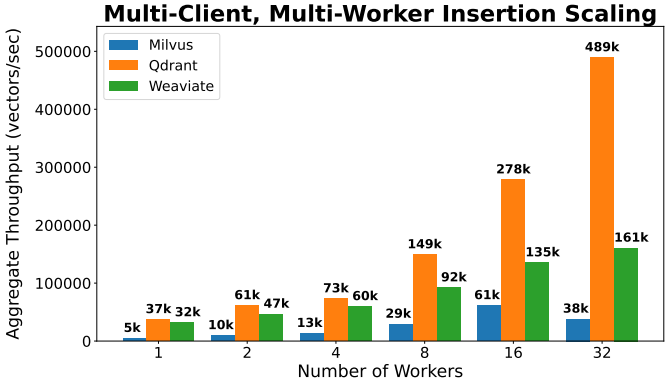

Fig. 4: Pes2o-VE: testing insertion scaling on Aurora.

As shown in Figure 4, Weaviate and Qdrant's insertion rate increases as the number of workers increases, achieving 160,551 vectors/s and 489,213 vectors/s, respectively. However, both systems scale sublinearly. Milvus's insertion rate scales up to a maximum of 61,344 vectors/second with 16 proxies and streaming nodes. Beyond that, throughput decreases. As an alternative to streaming-based ingestion, we also evaluated Milvus's bulk insertion utility, which bypasses the WAL and writes directly to object storage. Utilizing memory-backed MinIO-storage to maximize performance, we achieved an upload rate of 83,042 vectors/second.

4) Discussion: Notably, even the best-performing VDB's insertion throughput is well below the hardware capabilities of the HPC interconnect, which supports multiple 25 GB/s links (a theoretical maximum of roughly 2.6 million Pes2o-VE vectors per second per link) Although server-side components of the insertion pipeline (e.g., WAL writes, locking) are expected to limit achievable bandwidth, the observed gap suggests that enforcing per-insert consistency may be mismatched to the initial bulk ingestion phase of many VDB workloads. In this setting, such guarantees can unnecessarily constrain throughput. This observation motivates the need for ingestion modes that relax consistency guarantees during bulk loading to better utilize available system resources. Additionally, we observe that write amplification is a nontrivial cost. For example, a 95 GB subset of Pes2o-VE expands to 266 GB on Lustre using Milvus—an overhead that is tolerable at small scale but becomes prohibitive for large-scale workloads.

Lesson 3: Persistent storage latency dominates insertion performance: WAL latency and Lustre backpressure prevent VDBs from fully utilizing available bandwidth.

Lesson 4: HPC VDB deployments must treat write amplification as a key optimization metric, because additional storage overhead will quickly become intractable at scale.

B. Indexing

After data ingestion, each VDB advances to the next stage of the VDB lifecycle—HNSW indexing—where we examine how HPC resources influence performance. Section V-B1 studies the impact of GPU acceleration on indexing, since GPUs are a fundamental component of HPC systems. Section V-B2 evaluates the effect of many-core CPUs on indexing to expose scaling limitations. For clarity, note that we use the term *cores* to refer to the physical cores on a compute node's CPU, while we use the term *virtual cores* to refer to the node's hardware threads (e.g., Aurora has 104 cores and 208 virtual cores).

We select $M = 16$ and EF-C=100 as our HNSW parameters because they are commonly used baseline configurations in both the canonical literature [61] and popular ANN implementations [102], [103], providing a balanced trade-off between memory usage, construction cost, and recall. Note that for GPU-accelerated indexing, Milvus lacks a direct equivalent to HNSW. As a proxy, we employ its graph-based GPU-CAGRA index and configure its hyperparameters ($M = 16$, intermediate node edge limit during construction $= 64$) to approximate a similar construction process. Additionally, Weaviate does not provide GPU acceleration and is therefore excluded from GPU-based testing. Note that we omit reporting multinode index testing because it is an embarrassingly parallel workflow, and we observed near-linear scaling using Qdrant with the full Pes2o-VE dataset.

TABLE IV: Mean indexing time (seconds) across vector databases, methods, and HPC systems. Experiments were repeated three times; we observed minimal variance, and therefore omit standard deviation from the table.

VDB	Dataset	Size	SC1		SC2	
			CPU	GPU	CPU	GPU
Milvus	Pes2o-VE	1M	485	143	119	N/A
		5M	2398	718	368	N/A
		10M	4818	1541	691	N/A
	Yandex-T2I	1M	91	61	37	N/A
		5M	223	96	78	N/A
		10M	464	163	110	N/A
Qdrant	Pes2o-VE	1M	160	15	73	N/A
		5M	1077	61	420	N/A
		10M	1847	109	845	N/A
	Yandex-T2I	1M	20	5	8	N/A
		5M	109	22	58	N/A
		10M	219	45	131	N/A
Weaviate	Pes2o-VE	1M	1176	N/A	1726	N/A
		5M	6294	N/A	9779	N/A
		10M	13,402	N/A	19,457	N/A
	Yandex-T2I	1M	498	N/A	487	N/A
		5M	3001	N/A	3120	N/A
		10M	6415	N/A	5951	N/A

1) Cross-System CPU-GPU Indexing Analysis: We begin by evaluating indexing efficiency and the impact of GPU acceleration across varying data scales (1 million, 5 million, and 10 million) on Polaris and Aurora with Peso2-VE and Yandex-T2I. Table IV presents the results, with each experiment repeated three times and summarized using the mean. Our results show that indexing time is strongly influenced by both dataset size and embedding dimensionality. The Pes2o-VE dataset (2560 dimensions) consistently incurs significantly higher indexing costs than does the lower-dimensional Yandex-T2I dataset (200 dimensions). This trend is especially relevant because modern scientific and multimodal workloads increasingly rely on higher-dimensional embeddings. Interestingly, GPU acceleration exhibits a similar trend. Although it improves indexing performance across both datasets, at 10M points the gains are substantially larger for Pes2o-VE (10.04×) than for Yandex-T2I (3.86×), reflecting the increased computational intensity of higher-dimensional vectors and their suitability for GPU-based parallelism.

Focusing on CPU performance, Aurora's 104-core nodes reduce indexing time compared with Polaris's 32-core nodes, by an average of 4.08× and 2.59× for Pes2o-VE-10M and Yandex-T2I-10M with Qdrant and Milvus, respectively. On Polaris, Qdrant is consistently the fastest, whereas Milvus outperforms Qdrant on Aurora, and Weaviate trails both VDBs by a significant margin on both platforms. The divergence in

indexing time across VDBs highlights fundamentally different scaling behaviors, which we explore further in subsequent analysis.

Lesson 5: GPU-accelerated indexing delivers the greatest gains on high-dimensional datasets, where increased computational intensity and memory bandwidth demands better align with GPU architectures.

2) Impact of Core Count on Indexing: This section examines how indexing performance scales with virtual core count on Aurora. Using MPI-based binding, we restrict available virtual cores in powers of two, with an additional unrestricted configuration for execution without any binding. For Milvus and Weavite, we limit internal parallelism to match the number of virtual cores using `GOMAXPROCS`, while Qdrant automatically adjusts its thread-count to match the selected binding. To control compute-hour costs, we use a 10M subset of each dataset. Experiments start at 4 virtual cores, because smaller configurations resulted in timeouts that triggered runtime failures. Notably, Weaviate's transition from a flat to HNSW index is implemented as a batched sequential loop,[9] where each point in the batch is also inserted sequentially.[10] As a result, this phase does not benefit from additional cores, and we therefore omit Weaviate from Figure 5 and discuss its indexing behavior separately in the text.

As shown in Figure 5, increasing the number of virtual cores reduces indexing time for both Milvus and Qdrant, but with clear diminishing returns. For both datasets, the majority of gains are realized by 32–64 virtual cores, after which improvements taper off or reverse. Qdrant scales effectively at lower virtual core counts, achieving significant reductions in indexing time initially but limited gains past 32–64 virtual cores. This behavior is likely connected to its internal optimizer, which dynamically adjusts segment counts and indexing threads based on the detected virtual core count, creating varying thread-per-segment trade-offs. These dynamics are dataset dependent and warrant further study to fully understand their impact.

Milvus benefits from additional resources on Pes2o-VE, while on Yandex-T2I its performance peaks around 128 virtual cores. Notably, Milvus often completes its initial index faster than both Qdrant and Weaviate. However, because of the stabilization phase, during which background optimizations continue after the initial index is built, it falls beyond Qdrant with less than 128 and 64 virtual virtual cores with Pes2o-VE and Yandex-T2I, respectively. This creates a trade-off: while the initial index can be used to reduce wait time, we observe up to a 9.6× decrease in query throughput with the Pes2o-VE dataset when using the unoptimized index.

Weaviate's comparatively longer indexing time is a consequence of its internal indexing architecture. In dynamic index mode, Weaviate initially inserts vectors into a flat index and upgrades the collection to HNSW in a sequential loop once a threshold is reached. Moreover, because Weaviate maintains a single index per shard, it cannot build multiple indexes in parallel within a shard. This limits its ability to exploit the many-core architecture of modern HPC nodes and causes its indexing performance to lag behind the other VDBs. We selected Weaviate's dynamic index because it mirrors the design of both Qdrant and Weaviate; however, we also evaluate Weaviate's asynchronous indexing mechanism, which builds the HNSW index concurrently with data insertion using multiple threads. On the Yandex-T2I dataset, Weaviate's asynchronous indexing substantially reduced the total time to complete insertion and indexing with 10 million vectors from 6510 seconds to 709 seconds with comparable recall (0.7394 vs 0.738) and query throughput (8297 vs 7662 QPS).

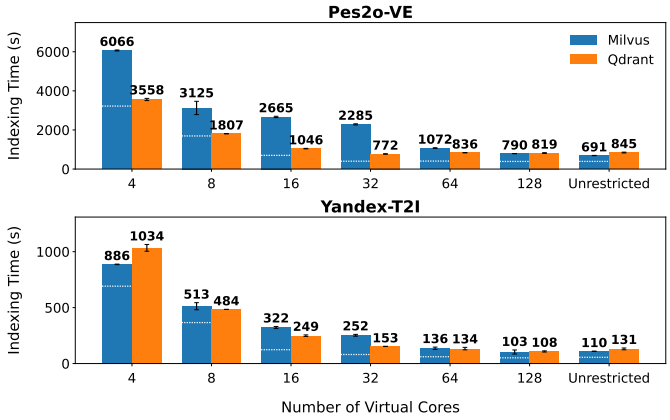

Fig. 5: Pes2o-VE-10M and Yandex-T2I=10M index core testing on Aurora. The white line in the bars indicates when Milvus finishes constructing an initial index. Weaviate is omitted because its index-building process is sequential.

C. Query

We now transition to the final stage of the VDB life cycle: querying. Section V-C1 examines hidden query recall-cost trade-offs introduced by the common practice of data segmentation, while section V-C2 tests single-node scalability, varying the number of virtual cores available. Section V-C3 assesses multinode scalability, testing with up to 256 workers on 64 compute nodes. Section V-D examines how performance changes when insertion and querying occur simultaneously. We evaluate an in-memory search scenario in which all data is loaded prior to query execution, and we place the VDBs on a memory-backed file system. Because Weaviate's Go client does not provide a method to issue batch queries, we issue batches of independent single-query requests using concurrent Goroutines. Unless otherwise noted, queries are sent sequentially in batches, and we set $efSearch = 64$, a commonly used mid-range HNSW value [61], [70] that balances latency and recall.

1) Single-Node Query Recall: Table V provides a comparison of each VDB's mean recall across three experiments using

[9]https://github.com/weaviate/weaviate/blob/v1.36.0/adapters/repos/db/vector/dynamic/index.go#L631

[10]https://github.com/weaviate/weaviate/blob/v1.36.0/adapters/repos/db/vector/hnsw/insert.go#L224

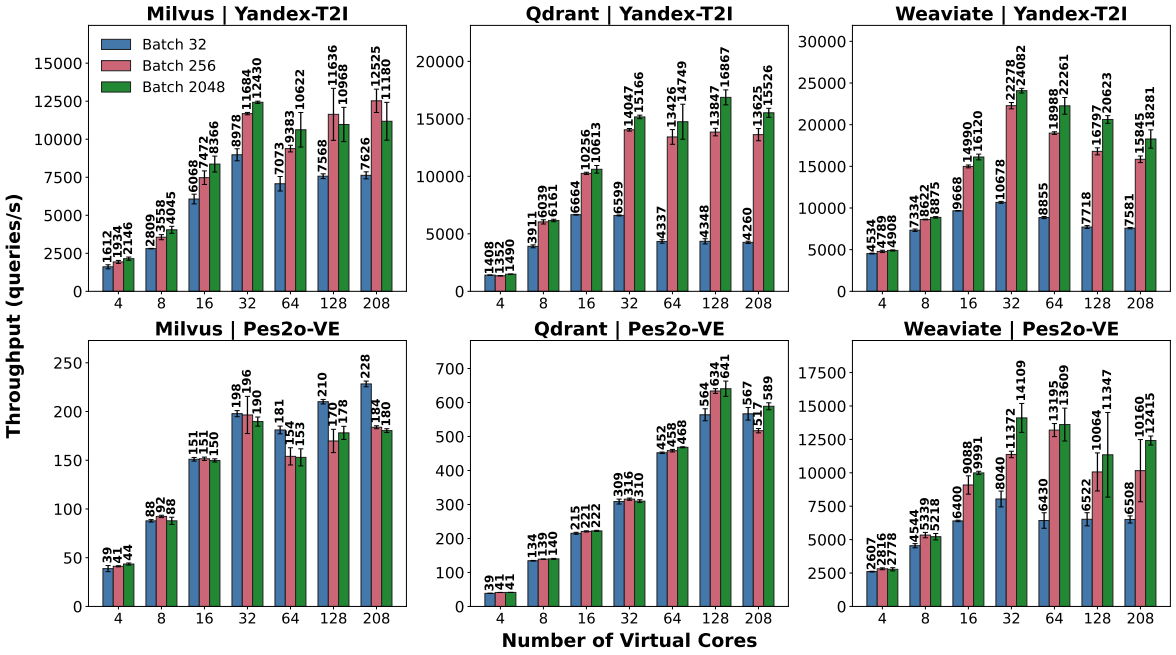

Fig. 6: Query performance with varied cores for Yandex-T2I-10M and Pes2o-VE-10M on Aurora.

TABLE V: Recall@10 comparison across VDBs for Yandex-T2I-10M and Pes2o-EV-10M datasets using full node-resources.

VDB	Yandex-T2I-10M	Pes2o-EV-10M
Weaviate	0.738	0.918
Qdrant	0.837	0.970
Milvus	0.876	0.982

all available cores, a batch size of 32, and 10M subsets of Pes2o-VE and Yandex-T2I. Notably, even with identical index parameters, the VDBs achieve different recall values: Milvus achieves the highest recall on Yandex-T2I, while Qdrant and Milvus achieve similar recall on Pes2o-VE. In both cases, Weaviate trails substantially behind the other two systems.

The gap between Milvus/Qdrant and Weaviate is caused by the two different approaches to single-shard indexing. Weaviate maintains a single HNSW index per shard, meaning the graph is constructed over the entire shard. In contrast, Qdrant and Milvus partition each shard into multiple segments, where each segment maintains an independent index that captures local data patterns. During search, Qdrant and Milvus query these segment-level HNSW graphs in parallel and then merge the segment-level top-k results into a final shard-wide top-k through reranking. In effect, this increases the thoroughness of the search within each subspace and expands the candidate set considered during reranking, improving recall at the cost of increased computation per query.

Follow-up experiments with Qdrant using a 1M subset of Pes2o-VE confirmed our hypothesis: increasing the number of segments from 1 to 8 improved recall from 0.962 to 0.9854 but also increased total search time from 4.19 s to 22.65s. Additionally, with full-node resources, we find that Qdrant and Milvus use different numbers of segments with Yandex-T2I-10M (8 vs 5), while with Pes2o-VE-10M, Milvus and Qdrant use 103 and 25 segments, respectively. In both cases, the VDB using more segments achieved higher recall; however, the Pes2o-VE results suggest that beyond some point, additional segments provide diminishing returns. These results show that each VDB encodes a largely unstudied cost-recall trade-off that extends beyond the standard HNSW parameters, motivating further investigation into optimal segment sizes.

Lesson 6: Modern VDBs make implicit cost-recall trade-offs by splitting large datasets into multiple segments, each with its own index. Utilizing more segments generally results in higher recall at the cost of increased computation per query.

2) Single-Node Query Core Scaling: To explore single-node query scaling, we vary virtual cores from 4 to the full node resources by powers of two in the same manner described in Section V-B2. We use 10M-vector subsets for tractability and evaluate batch sizes of 32, 256, and 2048, repeating all experiments three times to calculate mean and standard deviation.

a) Milvus: Figure 6 shows that Milvus exhibits inconsistent scaling behavior: in 4/6 configurations, Milvus reaches its peak performance with only 32 of the 208 available virtual cores. In all configurations, performance shows diminishing returns beyond this point, with the per-core marginal gain dropping substantially. Perf counters indicate this is due to increasing memory pressure. As the number of cores increases, memory stall cycles grow from 65.3% to 85.23%, leading to a drop in IPC (0.54→0.14) and retiring cycles (9.1%→ 3.6%).

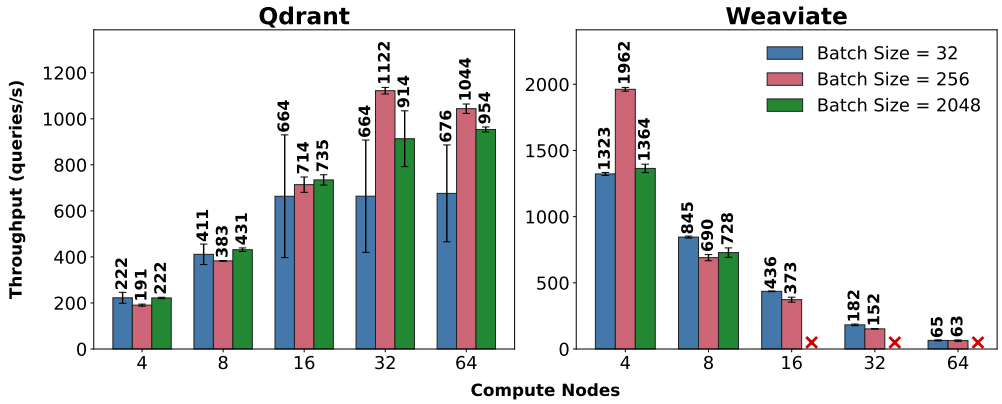

Fig. 7: Distributed query testing with the full Pes2o-VE dataset on Aurora. Note that because of runtime failures while using Lustre as their storage backend, we were unable to collect Milvus results (see Section V-C3). Each red "X" in the Weaviate graph indicates configurations where query testing repeatedly failed because of Weaviate intracluster networking failures.

b) Qdrant: Qdrant's scaling behavior appears more conventional at first glance, particularly with Pes2o-VE. However, Qdrant's scaling behavior is largely governed by internal optimization policies. Qdrant increases its internal search threads based on the detected core count and, inversely, changes settings that can cause the number of segments to decrease with more cores. This creates an effect where at lower core counts, there are significancy fewer segments and less threads-per-segment with Pes2o-VE, increasing search latency. For example, searching 50 segments with 32 virtual cores takes 98 ms, compared with 51 ms for 26 segments with 208 virtual cores (batch size = 32). In scenarios where the number of segments largely does not change across core counts—such as with the smaller Yandex-T2I dataset —we observe that scaling tapers after 32 cores. We see similar limited scaling behavior past 32 cores with Pes2o-VE when we fix the number of segments to 26 (the default selected with full node resources). This demonstrates that while Qdrant's segmentation model enables dynamic adaptation, it does not enable Qdrant to fully take advantage of many-core architectures.

c) Weaviate: Across batch sizes and datasets, Weaviate achieves its highest QPS by 32 virtual cores before its performance decreases with more resources. As shown in our Milvus analysis, HNSW traversal is memory-bound, with concurrent traversals contending for shared in-memory data structures. Furthermore, Weaviate's architectural design limits parallelism to the batch level. Unlike Qdrant and Milvus, which partition each shard and utilize multiple HNSW graphs, Weaviate maintains a single HNSW graph per shard, further limiting parallelism.

Lesson 7: QPS improves up to 32 cores but often plateaus or degrades beyond that, highlighting that current VDBs do not scale effectively on many-core HPC systems.

3) Multiworker Testing: After evaluating single-node scalability, we examine multinode strong scaling using the full Pes2o-VE dataset, scaling up to 64 compute nodes and testing batch sizes of 32, 256, and 2048. We use a single client that issues sequential batches, keeping the workload fixed while increasing the number of workers. To reduce compute-hour cost while still utilizing real-world scientific embeddings, we focus on the larger, higher-dimensional dataset Pes2o-VE. Note that while using a batch size of 2048 with more than 8 compute nodes, Weaviate's query operations repeatedly failed because of connection timeouts while contacting remote shards, preventing the collection of these results. Additionally, as will be discussed shortly, we were unable to scale Milvus to the full Peso2-VE dataset. All experiments are repeated three times, and we report the mean performance.

TABLE VI: Mean Recall@10 with the full Pes2o-VE dataset and a query batch size of 32 across compute-node counts for Qdrant and Weaviate.

Compute Nodes	Qdrant	Weaviate
4	0.946	0.910
8	0.954	0.920
16	0.962	0.934
32	0.943	0.935
64	0.959	0.931

Figure 7 presents the results of scaling from 4 to 64 compute nodes, while Table VI reports the achieved recall. Despite using identical HNSW parameters, the VDBs again exhibit a cost-recall trade-off, with Qdrant achieving slightly lower throughput but higher recall than Weaviate. Additionally, we observe that recall generally improves as the number of compute nodes increases. This mirrors the effect observed with segmentation: distributing the data across more compute nodes produces more independent HNSW graphs, improving candidate selection and increasing the number of candidates merged during the final reduction. Unlike increasing the number of local segments, however, the additional search work is distributed across more workers, allowing the cost of searching more graphs to be amortized through parallelism.

With a batch size of 32, Qdrant's performance improves from 4 to 16 compute nodes, peaking at a mean QPS of 664 be-

TABLE VII: Concurrent update and query performance. QPS = query vectors/second. Batch query latency and P99 latency are reported in milliseconds.

Data	System	Pattern	QPS	Mean Latency	P99 Latency
Pes2o	Milvus	Baseline	442.49 ± 4.34	72.23 ± 0.71	99.50 ± 2.46
	Milvus	Mixed	395.54 ± 8.69	81.05 ± 1.81	145.16 ± 17.51
	Weaviate	Baseline	$7{,}853.42 \pm 151.47$	4.07 ± 0.08	5.57 ± 0.11
	Weaviate	Mixed	$4{,}430.65 \pm 248.48$	7.41 ± 0.42	36.41 ± 1.56
	Qdrant	Baseline	751.42 ± 25.13	42.56 ± 1.40	59.15 ± 1.50
	Qdrant	Mixed	355.13 ± 13.33	90.12 ± 3.45	201.91 ± 20.18
Yandex-T2I	Milvus	Baseline	$11{,}812.39 \pm 55.47$	2.71 ± 0.01	3.63 ± 0.20
	Milvus	Mixed	$7{,}528.91 \pm 301.91$	4.27 ± 0.17	21.27 ± 0.26
	Weaviate	Baseline	$10{,}671.77 \pm 136.08$	3.00 ± 0.04	4.84 ± 0.14
	Weaviate	Mixed	$7{,}132.85 \pm 228.76$	4.53 ± 0.14	14.95 ± 0.47
	Qdrant	Baseline	$4{,}754.52 \pm 227.67$	6.74 ± 0.33	8.25 ± 1.44
	Qdrant	Mixed	$2{,}453.64 \pm 129.63$	13.05 ± 0.71	36.24 ± 4.53

fore tapering and eventually declining. With larger batch sizes of 256 and 2048, however, Qdrant continues scaling until 32 and 64 compute nodes, respectively. This suggests that batch size affects the balance between computation and communication overhead. All tested VDBs use a broadcast-gather query pattern where one worker broadcasts the query batch and then gathers/reduces the partial results. At small batch sizes, the per-query costs of communication, coordination, and reduction are amortized over relatively little computation, whereas larger batches better balance these costs. This is reflected in CPU utilization at 32 compute nodes: a batch size of 32 achieves consistently low CPU utilization of roughly 7%, whereas a batch size of 2048 produces burstier but substantially higher utilization, ranging from 30% to 70%. These effects are likely dataset-dependent, and practitioners need to consider the best balance of computation and communication for their use case.

In contrast to Qdrant, Weaviate's performance peaks at 4 compute nodes and then declines. Comparing the 4-node and 64-node configurations reveals significant workload imbalance that grows at scale. For the 4-node configuration, the merging worker exhibits CPU utilization of 28–50%, while a non-merging worker remains at 10–15%. This disparity suggests that non-merging workers have little useful work to perform, while the single merging worker increasingly becomes a bottleneck. This imbalance becomes more pronounced with 64 compute nodes: CPU utilization on the merging worker ranges from 5% to 31%, while a non-merging worker remains below 1% utilization during query execution. The observed trend reflects a poor communication-to-computation ratio in which coordination and merging overhead dominates useful computation.

Despite our best efforts, we were unable to successfully upload the full Pes2o-VE dataset to Milvus for query evaluation. The developer-recommended pure Lustre deployment proved too susceptible to performance variability, leading to runtime failures caused by failed attempts to create WAL entries for inserts (observed with 8 and 16 compute nodes). We also tested the standard MinIO-based deployment, using Lustre as a storage backend because of storage requirements. With the standard insertion path and the Milvus bulk-upload utility, the MinIO configuration exhibited Lustre incompatibilities that resulted in data loss and runtime failures.

Together, the multinode query results highlight key limitations of the current VDB broadcast-gather query pattern. As the number of workers increases, the single aggregating worker can become a bottleneck, limiting scalability. This effect is most pronounced in Weaviate, whose throughput declines beyond 4 nodes. Qdrant scales more effectively, but its performance remains well below linear scaling and is sensitive to batch size, indicating that coordination and reduction overheads still constrain distributed query performance.

Lesson 8: The standard VDB broadcast-gather query pattern exhibits limited scaling and places an unequal load on the single aggregating worker, motivating the design of new distributed query strategies.

D. Evaluating Mixed Read/Write Workloads

To provide a more complete picture of VDB performance, our final test evaluates a second workload pattern in which queries and insert requests are issued concurrently after an initial ingestion phase. We first insert 5 million vectors from Pes2o-VE and Yandex-T2I, allowing indexing to fully complete. This setup mimics a typical agentic workflow, where an agent begins with an initial knowledge base and later issues both queries and new inserts. We launch two independent clients: one sending queries and the other performing inserts (batch size = 32 for queries and inserts).

Because of differences in insertion rates across systems, the total amount of data ingested during the experiment varies between VDBs. As a result, direct comparisons across VDBs are less meaningful if a given query was executed on a larger corpus. To accommodate this, we evaluate each VBD's relative performance degradation under load by identifying the point at which the final insert occurs and then comparing performance against a baseline in which the same workload (insert, full indexing, and query) is executed without concurrency. This approach allows us to isolate the impact of overlapping inserts and queries, rather than conflating results with differences in dataset size across VDBs. We repeat all experiments three times and report the mean and standard deviation.

As shown in Table VII, concurrent insertion and querying degrade performance across all VDBs. The impact, however, differs between throughput, latency, and tail latency. Across the tested datasets, Milvus experiences the smallest average

throughput degradation, decreasing by 23.44%, compared with 38.37% for Weaviate and 50.57% for Qdrant, while latency increases by 34.99%, 102.74%, and 66.52% for Milvus, Qdrant, and Weaviate, respectively. In contrast, P99 latency increases much more sharply, rising by an average of 280.10% on Pes2o-VE and 344.77% on Yandex-T2I. This indicates that concurrent insertion has a disproportionate effect on the slowest queries, creating high-latency outliers.

The significant rise in P99 latency reflects how each VDB handles newly inserted data during query execution. Milvus and Qdrant place new vectors in unindexed segments that must be searched with an exhaustive scan, increasing query latency. Weaviate takes a different approach: rather than using exhaustive search for newly inserted vectors, it directly updates the active HNSW index while queries are running, creating contention for shared CPU and memory resources. Furthermore, as is typical of mixed read-write workloads, concurrent insertion introduces lock contention across the tested VDBs: queries must be prevented from reading segment or index state while it is being modified. This synchronization cost is especially visible in tail latency: although most queries may execute without substantial delay, queries that arrive during an active mutation can block, causing P99 latency to rise sharply.

Lesson 9: Concurrent inserts affect query P99 latency more than mean latency or throughput, suggesting that latency-sensitive applications may need specialized data structures to preserve query SLAs during ingestion.

VI. Discussion and Conclusion

Through large-scale deployment and evaluation of three VDBs on two HPC systems, several key insights emerge. We find that, compared with the cloud, HPC resources significantly improve indexing time and query throughput; however, they provide limited benefits for already low-latency queries (e.g., sub-ms). We observe that modern VDBs introduce a recall-cost trade-off by segmenting data across multiple indices. Between VDBs, we find that Milvus's indexing scales more effectively with core count than Qdrant, while Weaviate lags in indexing performance. Furthermore, our results show that, across VDBs, query performance frequently plateaus as cores increase and, in multinode settings, high communication costs limit multinode scaling. Beyond performance, we identify write amplification as a critical, underappreciated cost. Overall, we find that existing VDBs are frequently misaligned with HPC environments and are unable to fully harness HPC system resources.

Our findings indicate that researchers deploying VDBs should consider the influence of embedding geometry on potential HPC benefits, carefully configure their systems to avoid WAL bottlenecks during insertion, and explore different core counts to maximize query throughput. Additionally, our observations point to future research directions for VDBs. To overcome the significant underutilization of network resources during insertion, future VDBs can explore ingestion APIs with multiple data paths and tunable consistency–performance trade-offs for large-scale scientific datasets. Moreover, given that query performance exhibits limited scaling with core counts, our work shows that new threading and distribution models are required to effectively utilize HPC resources.[11]

Acknowledgment

This material is based upon work supported by the U.S. Department of Energy (DOE), Office of Science, Office of Advanced Scientific Computing Research, including the TIDES project at Argonne National Laboratory, under Contract No. DE-AC02-06CH11357. This research also used resources of the Argonne Leadership Computing Facility, a DOE Office of Science user facility. Additionally, this material is based upon work supported by the National Science Foundation Graduate Research Fellowship Program under Grant No. 2137424. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the National Science Foundation.

References

[1] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," 2013. [Online]. Available: https://doi.org/10.48550/arXiv.1301.3781

[2] J. Pennington, R. Socher, and C. Manning, "GloVe: Global vectors for word representation," in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, A. Moschitti, B. Pang, and W. Daelemans, Eds. Doha, Qatar: Association for Computational Linguistics, Oct. 2014, pp. 1532–1543. [Online]. Available: https://doi.org/10.3115/v1/D14-1162

[3] Q. Hu, X. Li, Z. Li, and Y. Zhang, "Generative AI of pinecone vector retrieval and retrieval-augmented generation architecture: Financial data-driven intelligent customer recommendation system," in *Proceedings of the 2025 2nd International Conference on Digital Economy and Computer Science*, ser. DECS '25. New York, NY, USA: Association for Computing Machinery, 2026, pp. 1227–1231. [Online]. Available: https://doi.org/10.1145/3785706.3785900

[4] O. B. Akhiiezer, O. A. Haluza, L. M. Lyubchyk, and V. Y. Sokol, "A curriculum recommendation system using a vector database for challenge-based learning," in *Journal of Physics Conference Series*, ser. Journal of Physics Conference Series, vol. 3105. IOP, Sep. 2025, p. 012023. [Online]. Available: https://doi.org/10.1088/1742-6596/3105/1/012023

[5] S. Y. Oh, A. Khan, I. Foster, and K. Chard, "SMURF: federated multimodal retrieval for scientific data via embedding alignment," in *2025 IEEE International Conference on eScience (eScience)*, 2025, pp. 452–459, https://doi.org/10.1109/eScience65000.2025.00091.

[6] B. Sarmah, B. Hall, R. Rao, S. Patel, S. Pasquali, and D. Mehta, "HybridRAG: integrating knowledge graphs and vector retrieval augmented generation for efficient information extraction," 2024. [Online]. Available: https://doi.org/10.1145/3677052.3698671

[7] W. Fan, Y. Ding, L. Ning, S. Wang, H. Li, D. Yin, T.-S. Chua, and Q. Li, "A survey on RAG meeting LLMs: Towards retrieval-augmented large language models," 2024. [Online]. Available: https://doi.org/10.1145/3637528.3671470

[8] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, "MemGPT: Towards LLMs as operating systems," 2024. [Online]. Available: https://doi.org/10.48550/arXiv.2310.08560

[9] D. Jiang, Y. Li, G. Li, and B. Li, "MAGMA: a multi-graph based agentic memory architecture for AI agents," 2026. [Online]. Available: https://doi.org/10.48550/arXiv.2601.03236

[10] W. Zhong, L. Guo, Q. Gao, H. Ye, and Y. Wang, "MemoryBank: Enhancing large language models with long-term memory," 2023. [Online]. Available: https://doi.org/10.48550/arXiv.2305.10250

[11] ChatGPT [104] was used to improve the grammar and phrasing of this work.

[11] W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, and Y. Zhang, "A-mem: Agentic memory for LLM agents," in *Advances in Neural Information Processing Systems*, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2502.12110

[12] B. Xia, N. Ramachandra, A. I. Wells, S. Habib, and J. Wise, "Multi-modal foundation model for cosmological simulation data," 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2510.07684

[13] G. Coulaud and D. Faranda, "TRAKNN: efficient trajectory aware spatiotemporal kNN for rare meteorological trajectory detection," 2026. [Online]. Available: https://doi.org/10.48550/arXiv.2603.02059

[14] Q. Yang, H. Zuo, R. Su, H. Su, T. Zeng, H. Zhou, R. Wang, J. Chen, Y. Lin, Z. Chen, and T. Tan, "Dual retrieving and ranking medical large language model with retrieval augmented generation," *Scientific Reports*, vol. 15, 05 2025. [Online]. Available: https://doi.org/10.1038/s41598-025-00724-w

[15] S. Ockerman, A. Gueroudji, S. Y. Oh, R. Underwood, N. Chia, K. Chard, R. Ross, and S. Venkataraman, "Exploring distributed vector databases performance on HPC platforms: A study with Qdrant," 2025. [Online]. Available: https://doi.org/10.1145/3731599.3767404

[16] M. Y. Lu, B. Chen, D. F. K. Williamson, R. J. Chen, K. Ikamura, G. Gerber, I. Liang, L. P. Le, T. Ding, A. V. Parwani, and F. Mahmood, "A foundational multimodal vision language ai assistant for human pathology," 2023. [Online]. Available: https://doi.org/10.48550/arXiv.2312.07814

[17] Z. Ji, S. Guo, Y. Qiao, and R. A. McDougal, "Automating literature screening and curation with applications to computational neuroscience," *Journal of the American Medical Informatics Association*, vol. 31, no. 7, pp. 1463–1470, 05 2024. [Online]. Available: https://doi.org/10.1093/jamia/ocae097

[18] V. Eyring, S. Bony, G. A. Meehl, C. A. Senior, B. Stevens, R. J. Stouffer, and K. E. Taylor, "Overview of the coupled model intercomparison project phase 6 (CMIP6) experimental design and organization," *Geoscientific Model Development*, vol. 9, no. 5, pp. 1937–1958, 2016, https://doi.org/10.5194/gmd-9-1937-2016.

[19] S. Fairley, E. Lowy-Gallego, E. Perry, and P. Flicek, "The International Genome Sample Resource (IGSR) collection of open human genomic variation resources," *Nucleic Acids Research*, vol. 48, no. D1, pp. D941–D947, 10 2019. [Online]. Available: https://doi.org/10.1093/nar/gkz836

[20] F. Cappello, R. Underwood, Y. Alexeev, A. Baker, E. Bozdağ, M. Burtscher, K. Chard, S. Di, K. G. Felker, P. C. O'Grady, H. Guo, Y. Huang, P. Jiang, S. Jin, P. Johansson, S. Li, X. Liang, E. Lindahl, P. Lindstrom, Z. Lukić, M. Lundborg, D. Lykov, M. Nagaso, K. Sato, A. Singh, S. W. Son, S. Song, W. Tang, D. Tao, J. Tian, K. Yoshii, and K. Zhao, "What to support when you're compressing: The state of practice gaps and opportunities for scientific data compression," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. St. Louis MO USA: ACM, Nov. 2025, pp. 1966–1979. [Online]. Available: https://doi.org/10.1145/3712285.3759856

[21] K. K. Yang, Z. Wu, C. N. Bedbrook, and F. H. Arnold, "Learned protein embeddings for machine learning," *Bioinformatics*, vol. 34, no. 15, pp. 2642–2648, 03 2018. [Online]. Available: https://doi.org/10.1093/bioinformatics/bty178

[22] Y. S. Ko, J. Parkinson, and W. Wang, "Scalable embedding fusion with protein language models: insights from benchmarking text-integrated representations," *Briefings in Bioinformatics*, vol. 27, no. 1, p. bbag014, 01 2026. [Online]. Available: https://doi.org/10.1093/bib/bbag014

[23] S. Ali, P. Chourasia, and M. Patterson, "When protein structure embedding meets large language models," *Genes*, vol. 15, no. 1, 2024. [Online]. Available: https://doi.org/10.3390/genes15010025

[24] J. Wang, X. Yi, R. Guo, H. Jin, P. Xu, S. Li, X. Wang, X. Guo, C. Li, X. Xu, K. Yu, Y. Yuan, Y. Zou, J. Long, Y. Cai, Z. Li, Z. Zhang, Y. Mo, J. Gu, R. Jiang, Y. Wei, and C. Xie, "Milvus: A purpose-built vector data management system," in *Proceedings of the 2021 International Conference on Management of Data*, ser. SIGMOD '21. New York, NY, USA: Association for Computing Machinery, 2021, pp. 2614–2627. [Online]. Available: https://doi.org/10.1145/3448016.3457550

[25] Vespa Team, "Vespa," 2026. [Online]. Available: https://vespa.ai/

[26] Qdrant Team, "Qdrant," 2026. [Online]. Available: https://qdrant.tech/

[27] Vald Team, "Vald," 2026. [Online]. Available: https://vald.vdaas.org/

[28] Weaviate Team, "Weaviate," 2026. [Online]. Available: https://weaviate.io/

[29] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," 2017, https://doi.org/10.48550/arXiv.1702.08734.

[30] Z. Li, W. Ding, S. Huang, Z. Wang, Y. Lin, K. Wu, Y. Park, and J. Chen, "Cloud-native vector search: A comprehensive performance analysis," 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2511.14748

[31] J. R. Lange and et al., "Evaluating the cloud for capability class leadership workloads," Oak Ridge National Laboratory, Technical Report ORNL/TM-2023/3083, September 2023. [Online]. Available: https://doi.org/10.2172/2000306

[32] V. Sochat, D. Milroy, A. Sarkar, A. Marathe, and T. Patki, "Usability evaluation of cloud for HPC applications," 2025. [Online]. Available: https://doi.org/10.1145/3731599.3767353

[33] V. Munhoz, A. Bonfils, M. Castro, and O. Mendizabal, "A performance comparison of hpc workloads on traditional and cloud-based HPC clusters," in *2023 International Symposium on Computer Architecture and High Performance Computing Workshops (SBAC-PADW)*, 2023, pp. 108–114, https://doi.org/10.1109/SBAC-PADW60351.2023.00026.

[34] R. Latham, R. B. Ross, P. Carns, S. Snyder, K. Harms, K. Velusamy, P. Coffman, and G. McPheeters, "Initial experiences with DAOS object storage on Aurora," in *Proceedings of the SC '24 Workshops of the International Conference on High Performance Computing, Network, Storage, and Analysis*, ser. SC-W '24. IEEE Press, 2025, pp. 1304–1310. [Online]. Available: https://doi.org/10.1109/SCW63240.2024.00171

[35] J. Kwack, C. Bertoni, U. Unnikrishnan, R. Balin, K. Hossain, Y. Ghadar, T. J. Williams, A. Bagusetty, M. Thavappiragasam, V. Hatanpää, A. Vasan, J. Tramm, and S. Parker, "AI and HPC applications on leadership computing platforms: Performance and scalability studies," in *2025 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2025, pp. 210–222. [Online]. Available: https://doi.org/10.1109/IPDPS64566.2025.00027

[36] B. Homerding, B. Lenard, C. Blackworth, C. Holohan, A. Kulyavtsev, G. McPheeters, E. Pershy, P. Rich, D. Waldron, M. Zhang, K. Harms, T. Leggett, and W. Allcock, "Polaris and acceptance testing." CUG, 2023, https://cug.org/proceedings/cug2023_proceedings/includes/files/pap109s2-file1.pdf.

[37] Z. Liang, J. Lombardi, M. Chaarawi, and M. Hennecke, "DAOS: A scale-out high performance storage stack for storage class memory," in *Supercomputing Frontiers: 6th Asian Conference, SCFA 2020, Singapore, February 24–27, 2020, Proceedings*. Berlin, Heidelberg: Springer-Verlag, 2020, pp. 40–54. [Online]. Available: https://doi.org/10.1007/978-3-030-48842-0_3

[38] A. B. Yandex and V. Lempitsky, "Efficient indexing of billion-scale datasets of deep descriptors," in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 2055–2063. [Online]. Available: https://doi.org/10.1109/CVPR.2016.226

[39] H. Jégou, M. Douze, and C. Schmid, "Product quantization for nearest neighbor search," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 33, no. 1, pp. 117–128, 2011. [Online]. Available: https://doi.org/10.1109/TPAMI.2010.57

[40] Kumar Shivendu and Nirant Kasliwal, "dbpedia-entities-openai-1m," 2023, https://huggingface.co/datasets/KShivendu/dbpedia-entities-openai-1M.

[41] X. Yan, N. Hudson, H. Park, D. Grzenda, J. G. Pauloski, M. Schwarting, H. Pan, H. Harb, S. Foreman, C. Knight, T. Gibbs, K. Chard, S. Chaudhuri, E. Tajkhorshid, I. Foster, M. Moosavi, L. Ward, and E. A. Huerta, "MOFA: discovering materials for carbon capture with a GenAI- and simulation-based workflow," 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2501.10651

[42] T. Hellert, J. Montenegro, and A. Sulc, "Osprey: Production-ready agentic AI for safety-critical control systems," 2025. [Online]. Available: https://doi.org/10.1063/5.0306302

[43] M. Aumüller and M. Ceccarello, "The role of local intrinsic dimensionality in benchmarking nearest neighbor search," 2019. [Online]. Available: https://doi.org/10.48550/arXiv.1907.07387

[44] M. E. Houle, E. Schubert, and A. Zimek, "On the correlation between local intrinsic dimensionality and outlierness," in *Similarity Search and Applications: 11th International Conference, SISAP 2018, Lima, Peru, October 7–9, 2018, Proceedings*. Berlin, Heidelberg: Springer-Verlag, 2018, pp. 177–191, https://dl.acm.org/doi/10.1007/978-3-030-02224-2_14.

[45] O. P. Elliott and J. Clark, "The impacts of data, ordering, and intrinsic dimensionality on recall in hierarchical navigable small worlds," in *Proceedings of the 2024 ACM SIGIR International Conference on*

Theory of Information Retrieval, ser. ICTIR '24. ACM, Aug. 2024, p. 25–33. [Online]. Available: https://doi.org/10.1145/3664190.3672512

[46] P. Braam, "The Lustre storage architecture," 2019. [Online]. Available: https://doi.org/10.48550/arXiv.1903.01955

[47] Y. Zhang, M. Li, D. Long, X. Zhang, H. Lin, B. Yang, P. Xie, A. Yang, D. Liu, J. Lin, F. Huang, and J. Zhou, "Qwen3 embedding: Advancing text embedding and reranking through foundation models," *arXiv preprint arXiv:2506.05176*, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2506.05176

[48] C. Lee, R. Roy, M. Xu, J. Raiman, M. Shoeybi, B. Catanzaro, and W. Ping, "NV-Embed: Improved techniques for training LLMs as generalist embedding models," 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2405.17428

[49] M. Muja and D. G. Lowe, "Scalable nearest neighbor algorithms for high dimensional data," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 36, pp. 2227–2240, 2014. [Online]. Available: https://doi.org/10.1109/TPAMI.2014.2321376

[50] A. Gionis, P. Indyk, and R. Motwani, "Similarity search in high dimensions via hashing," in *Proceedings of the 25th International Conference on Very Large Data Bases*, ser. VLDB '99. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 1999, p. 518–529. [Online]. Available: https://dl.acm.org/doi/10.5555/645925.671516

[51] P. N. Yianilos, "Data structures and algorithms for nearest neighbor search in general metric spaces," in *Proceedings of the Fourth Annual ACM-SIAM Symposium on Discrete Algorithms*, ser. SODA '93. USA: Society for Industrial and Applied Mathematics, 1993, pp. 311–321. [Online]. Available: https://dl.acm.org/doi/10.5555/313559.313789

[52] P. Indyk and R. Motwani, "Approximate nearest neighbors: towards removing the curse of dimensionality," in *Proceedings of the Thirtieth Annual ACM Symposium on Theory of Computing*, ser. STOC '98. New York, NY, USA: Association for Computing Machinery, 1998, pp. 604–613. [Online]. Available: https://doi.org/10.1145/276698.276876

[53] J. L. Bentley, "Multidimensional binary search trees used for associative searching," *Commun. ACM*, vol. 18, no. 9, pp. 509–517, Sep. 1975. [Online]. Available: https://doi.org/10.1145/361002.361007

[54] A. Guttman, "R-trees: a dynamic index structure for spatial searching," in *Proceedings of the 1984 ACM SIGMOD International Conference on Management of Data*, ser. SIGMOD '84. New York, NY, USA: Association for Computing Machinery, 1984, pp. 47–57. [Online]. Available: https://doi.org/10.1145/602259.602266

[55] J. Mackenzie, S. MacAvaney, A. Mallia, and M. Siedlaczek, "Efficient in-memory inverted indexes: Theory and practice," in *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*, ser. SIGIR '25. New York, NY, USA: Association for Computing Machinery, 2025, pp. 4102–4105. [Online]. Available: https://doi.org/10.1145/3726302.3731688

[56] A. Andoni and P. Indyk, "Near-optimal hashing algorithms for approximate nearest neighbor in high dimensions," *Commun. ACM*, vol. 51, no. 1, pp. 117–122, Jan. 2008. [Online]. Available: https://doi.org/10.1145/1327452.1327494

[57] O. Jafari, P. Maurya, P. Nagarkar, K. M. Islam, and C. Crushev, "A survey on locality sensitive hashing algorithms and their applications," 2021. [Online]. Available: https://doi.org/10.48550/arXiv.2102.08942

[58] S. J. Subramanya, Devvrit, R. Kadekodi, R. Krishaswamy, and H. V. Simhadri, "Diskann: Fast accurate billion-point nearest neighbor search on a single node," in *NeurIPS 2019*, November 2019. [Online]. Available: https://dl.acm.org/doi/abs/10.5555/3454287.3455520

[59] C. Fu, C. Xiang, C. Wang, and D. Cai, "Fast approximate nearest neighbor search with the navigating spreading-out graph," 2025. [Online]. Available: https://doi.org/10.14778/3303753.3303754

[60] H. Ootomo, A. Naruse, C. Nolet, R. Wang, T. Feher, and Y. Wang, "CAGRA: highly parallel graph construction and approximate nearest neighbor search for GPUs," 2024. [Online]. Available: https://doi.org/10.48550/arXiv.2308.15136

[61] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," 2018, https://doi.org/10.48550/arXiv.1603.09320.

[62] Rapidsai, "Rapidsai/raft: RAFT contains fundamental widely-used algorithms and primitives for data science, graph and machine learning." 2022. [Online]. Available: https://github.com/rapidsai/raft

[63] I. Stoica, R. Morris, D. Karger, M. F. Kaashoek, and H. Balakrishnan, "Chord: A scalable peer-to-peer lookup service for internet applications," in *Proceedings of the 2001 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communications*, ser. SIGCOMM '01. New York, NY, USA: Association for Computing Machinery, 2001, pp. 149–160. [Online]. Available: https://doi.org/10.1145/383059.383071

[64] A. Cuzzocrea, "Vector databases for modelling, managing and querying big scientific data: Models, issues, paradigms," in *Proceedings of the 37th International Conference on Scalable Scientific Data Management*, ser. SSDBM '25. New York, NY, USA: Association for Computing Machinery, 2025, https://doi.org/10.1145/3733723.3742469.

[65] T. Taipalus, "Vector database management systems: Fundamental concepts, use-cases, and current challenges," *Cognitive Systems Research*, vol. 85, p. 101216, 2024. [Online]. Available: https://doi.org/10.48550/arXiv.2309.11322

[66] J. J. Pan, J. Wang, and G. Li, "Survey of vector database management systems," 2023. [Online]. Available: https://doi.org/10.48550/arXiv.2310.14021

[67] Y. Han, C. Liu, and P. Wang, "A comprehensive survey on vector database: Storage and retrieval technique, challenge," *arXiv preprint arXiv:2310.11703*, 2023. [Online]. Available: https://doi.org/10.48550/arXiv.2310.11703

[68] M. Shen, M. Umar, K. Maeng, G. E. Suh, and U. Gupta, "Towards understanding systems trade-offs in retrieval-augmented generation model inference," 2024. [Online]. Available: https://doi.org/10.48550/arXiv.2412.11854

[69] M. Wang, X. Xu, Q. Yue, and Y. Wang, "A comprehensive survey and experimental comparison of graph-based approximate nearest neighbor search," *Proc. VLDB Endow.*, vol. 14, no. 11, p. 1964–1978, Jul. 2021. [Online]. Available: https://doi.org/10.14778/3476249.3476255

[70] M. Aumüller, E. Bernhardsson, and A. Faithfull, "ANN-Benchmarks: a benchmarking tool for approximate nearest neighbor algorithms," 2018. [Online]. Available: https://doi.org/10.48550/arXiv.1807.05614

[71] G. Pandit, M. Roder, and A.-C. Ngonga Ngomo, "Evaluating approximate nearest neighbour search systems on knowledge graph embeddings," in *The Semantic Web: 22nd European Semantic Web Conference, ESWC 2025, Portoroz, Slovenia, June 1–5, 2025, Proceedings, Part I*. Berlin, Heidelberg: Springer-Verlag, 2025, pp. 59–76. [Online]. Available: https://doi.org/10.1007/978-3-031-94575-5_4

[72] S. Li, Y. Zhou, Y. Xu, K. Chen, D. Waddington, S. Sundararaman, H. Franke, and J. Huang, "RAGPerf: An end-to-end benchmarking framework for retrieval-augmented generation systems," 2026. [Online]. Available: https://doi.org/10.48550/arXiv.2603.10765

[73] Weaviate, "ANN benchmark," https://docs.weaviate.io/weaviate/benchmarks/ann, 2024, weaviate documentation; accessed 2026-03.

[74] Qdrant, "Vector search benchmarks," https://qdrant.tech/benchmarks/, 2024, accessed: 2026-03.

[75] Tian Min, "Announcing VDBBench 1.0: open-source vector database benchmarking with your real-world production workloads," https://milvus.io/blog/vdbbench-1-0-benchmarking-with-your-real-world-production-workloads.md, July 2025, milvus Blog; accessed 2026-03.

[76] Y. Xu, Q. Zhang, Q. Chen, B. Lu, M. Li, P. Adams, M. Li, Z. Li, J. Liu, C. Li, and F. Yang, "Scalable distributed vector search via accuracy preserving index construction," 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2512.17264

[77] X. Zhi, M. Chen, X. Yan, B. Lu, H. Li, Q. Zhang, Q. Chen, and J. Cheng, "Towards efficient and scalable distributed vector search with RDMA," 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2507.06653

[78] Q. Xu, F. Zhang, C. Li, L. Cao, Z. Chen, J. Zhai, and X. Du, "HARMONY: A scalable distributed vector database for high-throughput approximate nearest neighbor search," *arXiv preprint arXiv:2506.14707*, 2025. [Online]. Available: https://doi.org/10.1145/3749167

[79] G. Hu, S. Cai, T. T. A. Dinh, Z. Xie, C. Yue, G. Chen, and B. C. Ooi, "HAKES: scalable vector database for embedding search service," *Proceedings of the VLDB Endowment*, vol. 18, no. 9, p. 3049–3062, May 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2505.12524

[80] K. Heitmann, H. Finkel, A. Pope, V. Morozov, N. Frontiere, S. Habib, E. Rangel, T. Uram, D. Korytov, H. Child, S. Flender, J. Insley, and S. Rizzi, "The outer rim simulation: A path to many-core supercomputers," *The Astrophysical Journal Supplement Series*, vol. 245, no. 1, p. 16, Nov. 2019. [Online]. Available: https://doi.org/10.48550/arXiv.1904.11970

[81] D. Wang, C. Wang, Q. Cao, P. Schwartz, F. Yuan, J. Krishna, D. Wu, D. Ricciuto, P. Thornton, S.-C. Kao, M. Thornton, and K. Mohror, "Kilometer-scale E3SM land model simulation over North America," 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2501.11141

[82] H. V. Simhadri, G. Williams, M. Aumüller, M. Douze, A. Babenko, D. Baranchuk, Q. Chen, L. Hosseini, R. Krishnaswamy, G. Srinivasa, S. J. Subramanya, and J. Wang, "Results of the NeurIPS'21 challenge on billion-scale approximate nearest neighbor search," in *Proceedings of the NeurIPS 2021 Competitions and Demonstrations Track*, ser. Proceedings of Machine Learning Research, vol. 176. PMLR, 2022, pp. 177–189.

[83] W. Chen, H. Liu, Y. Deng, L. Xiang, L. Huang, G. Li, and B. Tang, "AlayaLaser: Efficient index layout and search strategy for large-scale high-dimensional vector similarity search," 2026. [Online]. Available: https://doi.org/10.48550/arXiv.2602.23342

[84] S. Williams, A. Waterman, and D. Patterson, "Roofline: an insightful visual performance model for multicore architectures," *Commun. ACM*, vol. 52, no. 4, pp. 65–76, Apr. 2009. [Online]. Available: https://doi.org/10.1145/1498765.1498785

[85] OpenAI, "Introducing text and code embeddings," https://openai.com/index/introducing-text-and-code-embeddings/, 2022, accessed: 2025-02.

[86] R. D. Olson, R. Assaf, T. Brettin, N. Conrad, C. Cucinell, J. J. Davis, D. M. Dempsey, A. Dickerman, E. M. Dietrich, R. W. Kenyon, M. Kuscuoglu, E. J. Lefkowitz, J. Lu, D. Machi, C. Macken, C. Mao, A. Niewiadomska, M. Nguyen, G. J. Olsen, J. C. Overbeek, B. Parrello, V. Parrello, J. S. Porter, G. D. Pusch, M. Shukla, I. Singh, L. Stewart, G. Tan, C. Thomas, M. VanOeffelen, V. Vonstein, Z. S. Wallace, A. S. Warren, A. R. Wattam, F. Xia, H. Yoo, Y. Zhang, C. M. Zmasek, R. H. Scheuermann, and R. L. Stevens, "Introducing the bacterial and viral bioinformatics resource center (bv-brc): a resource combining patric, ird and vipr," *Nucleic Acids Research*, vol. 51, no. D1, pp. D678–D689, 11 2022. [Online]. Available: https://doi.org/10.1093/nar/gkac1003

[87] L. Soldaini and K. Lo, "peS2o (pretraining efficiently on S2ORC) dataset," Tech. Rep., 2023, https://github.com/allenai/pes2o.

[88] C. Schuhmann, R. Vencu, R. Beaumont, R. Kaczmarczyk, C. Mullis, A. Katta, T. Coombes, J. Jitsev, and A. Komatsuzaki, "LAION-400M: open dataset of CLIP-filtered 400 million image-text pairs," 2021. [Online]. Available: https://doi.org/10.48550/arXiv.2111.02114

[89] N. Salsabilla and K. Wiharja, "Implementation of semantic search based on vector database for personal documents," in *2025 International Conference on Advancement in Data Science, E-learning and Information System (ICADEIS)*, 2025, pp. 1–6. [Online]. Available: https://doi.org/10.1109/ICADEIS65852.2025.10933119

[90] H. Mentzingen, N. António, F. Bacao, and M. Cunha, "Textual similarity for legal precedents discovery: Assessing the performance of machine learning techniques in an administrative court," *International Journal of Information Management Data Insights*, vol. 4, no. 2, p. 100247, 2024. [Online]. Available: https://doi.org/10.1016/j.jjimei.2024.100247

[91] Harvard Library Innovation Lab, "COLD Cases: Collaborative open legal data (U.S. court decisions) dataset," https://huggingface.co/datasets/harvard-lil/cold-cases, 2024, accessed: 2026-02-25. [Online]. Available: https://huggingface.co/datasets/harvard-lil/cold-cases

[92] A. Singh, J. C. Chang, C. Anastasiades, D. Haddad, A. Naik, A. Tanaka, A. Zamarron, C. Nguyen, J. D. Hwang, J. Dunkleberger, M. Latzke, S. Rao, J. Lochner, R. Evans, R. Kinney, D. S. Weld, D. Downey, and S. Feldman, "AI2 Scholar QA: organized literature synthesis with attribution," 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2504.10861

[93] K. Lo, L. L. Wang, M. Neumann, R. Kinney, and D. Weld, "S2ORC: The semantic scholar open research corpus," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, D. Jurafsky, J. Chai, N. Schluter, and J. Tetreault, Eds. Online: Association for Computational Linguistics, Jul. 2020, pp. 4969–4983. [Online]. Available: https://doi.org/10.48550/arXiv.1911.02782

[94] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," *Proceedings of NIPS'20*, pp. 9459–9474, 2020. [Online]. Available: https://doi.org/10.48550/arXiv.2005.11401

[95] A. Maharana, D.-H. Lee, S. Tulyakov, M. Bansal, F. Barbieri, and Y. Fang, "Evaluating very long-term conversational memory of LLM agents," 2024. [Online]. Available: https://doi.org/10.48550/arXiv.2402.17753

[96] Argonne National Laboratory, "Argonne resilience ai assistant (araia)," https://www.anl.gov/dis/argonne-resilience-ai-assistant, 2025, accessed: 2026-02-25.

[97] P. Covington, J. Adams, and E. Sargin, "Deep neural networks for YouTube recommendations," in *Proceedings of the 10th ACM Conference on Recommender Systems*, ser. RecSys '16. New York, NY, USA: Association for Computing Machinery, 2016, pp. 191–198. [Online]. Available: https://doi.org/10.1145/2959100.2959190

[98] E. Levina and P. J. Bickel, "Maximum likelihood estimation of intrinsic dimension," in *Proceedings of the 18th International Conference on Neural Information Processing Systems*, ser. NIPS'04. Cambridge, MA, USA: MIT Press, 2004, pp. 777–784. [Online]. Available: https://dl.acm.org/doi/10.5555/2976040.2976138

[99] Z. Chen, R. Zhang, X. Zhao, X. Cheng, and X. Zhou, "Exploring the meaningfulness of nearest neighbor search in high-dimensional space," 2024. [Online]. Available: https://doi.org/10.48550/arXiv.2410.05752

[100] N. Muennighoff, N. Tazi, L. Magne, and N. Reimers, "MTEB: massive text embedding benchmark," 2023. [Online]. Available: https://doi.org/10.48550/arXiv.2210.07316

[101] B. Bhalla, H. Fan, N. Chen, and T. Y. YU, "Higher embedding dimension creates a stronger world model for a simple sorting task," 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2510.18315

[102] Y. A. Malkov and D. A. Yashunin, "Hnswlib - fast approximate nearest neighbor search," 2026. [Online]. Available: https://github.com/nmslib/hnswlib

[103] OpenSearch Project, "k-NN Index — OpenSearch Documentation," 2024, accessed: 2026. [Online]. Available: https://docs.opensearch.org/latest/mappings/supported-field-types/knn-vector/

[104] OpenAI, "ChatGPT," https://chat.openai.com, 2026, large language model, accessed April 2026.